МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федерального государственного бюджетного образовательного учреждения высшего образования

«РОССИЙСКИЙ ЭКОНОМИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ Г.В.ПЛЕХАНОВА»

Пермский институт (филиал)

Практическая работа по дисциплине «Поддержка и тестирование программных модулей»

по теме: «Создание программы “Управление складом”»

Руководитель: преподаватель поддержки и тестирования программных модулей

Берестов Д.Б.

«___» _____2026г.

Исполнитель: студент 4 курса

ИПо-41

Андреев Е.А.

«___» _____2026г.

Пермь, 2026г

Оглавление

Введение ... 2
1. Цель и задачи работы ... 3
2. Архитектура решения ... 3
3. Структура решения ... 4
4. Архитектура приложения ... 5
4.1 Слой данных - WarehouseData ... 5
4.2 Слой представления – Warehouse ... 5
5. Модели данных ... 6
6. Контекст данных – ApplicationContext ... 8
7. Сервисы ... 9
8. Unit-тесты ... 12
9. Пользовательский интерфейс ... 14
9.1 Единый стиль - ResourceDictionary ... 14
9.2 Навигация ... 14
9.3 RoutedCommands ... 15
10. Описание модулей ... 16
10.1 Модуль организаций ... 16
10.2 Модуль складов ... 17
10.3 Модуль товаров ... 17
10.4 Модуль накладных ... 19
11. Валидация данных ... 20
12. Хранение данных ... 21
Заключение ... 22

Введение

В современных информационных системах автоматизация складского учета играет важную роль в организации работы предприятия. Эффективное управление справочными данными позволяет обеспечить корректный учет товаров, контроль складских операций и удобное хранение информации о юридических лицах и складах.

В рамках данной практической работы разрабатывается модуль системы «Управление складом», предназначенный для ведения и редактирования основных справочников. Особое внимание уделяется

созданию удобного и понятного пользовательского интерфейса, который обеспечивает быстрый доступ к данным и упрощает работу пользователя.

Разработка выполняется с использованием языка программирования C# и технологии WPF, что позволяет создать современное настольное приложение с графическим интерфейсом. Для организации структуры проекта рекомендуется применение архитектурного подхода MVVM, обеспечивающего разделение логики, интерфейса и данных приложения.

1. Цель и задачи работы

Данная работа представляет собой десктоп-приложение для управления складским хозяйством, разработанное на языке C# с использованием технологии WPF (Windows Presentation Foundation). Приложение является аналогом простой учётной системы типа 1С и позволяет вести учёт организаций, складов, товаров и накладных.

Цель работы - разработать полноценное desktop-приложение с архитектурой MVVM, реализовать CRUD-операции через сервисы, покрыть логику Unit-тестами и создать удобный пользовательский интерфейс.

2. Архитектура решения

Разрабатываемая система «Управление складом» построена на основе архитектурного подхода MVVM (Model–View–ViewModel), который обеспечивает разделение пользовательского интерфейса, логики приложения и данных. Такой подход упрощает поддержку проекта, делает код более структурированным и удобным для дальнейшего расширения. Архитектура приложения включает два основных компонента:

Model - отвечает за хранение и обработку данных системы. В данном слое располагаются классы, описывающие сущности предметной области: юридические лица, склады и товары.

View - представляет пользовательский интерфейс приложения, созданный с использованием технологии WPF и языка разметки XAML. View обеспечивает отображение данных и взаимодействие пользователя с системой. Для организации интерфейса используются окна, страницы, элементы управления WPF и ресурсы оформления. Применение ResourceDictionary позволяет хранить стили и шаблоны интерфейса отдельно от основной логики приложения, обеспечивая единый внешний вид системы.

Структура проекта разделена на отдельные модули, отвечающие за управление юридическими лицами, складами и товарами, что обеспечивает удобство разработки и масштабируемость приложения.

3. Структура решения

Решение состоит из трёх проектов:

- Warehouse - WPF приложение (UI)
- WarehouseData - Библиотека классов (логика)
- WarehouseDataTests - Проект Unit-тестов

Разделение на проекты сделано намеренно - логика приложения полностью отделена от интерфейса. Это позволяет тестировать сервисы без запуска окон, а также при необходимости заменить UI не трогая бизнеслогику.

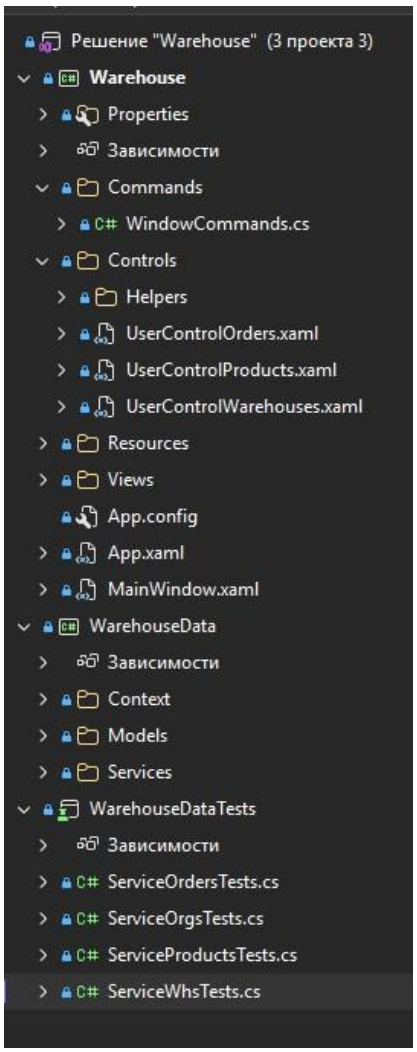


Рисунок 1 - структура трёх проектов в Visual Studio

4. Архитектура приложения

4.1 Слой данных - WarehouseData

Библиотека классов (WarehouseData) содержит всё что не зависит от UI:

- Models - классы-модели данных
- Context - хранилище данных (ApplicationContext)
- Services - сервисы с CRUD-операциями

4.2 Слой представления – Warehouse

WPF проект (Warehouse) содержит всё что относится к интерфейсу:

- Views - окна приложения

- Controls - пользовательские элементы управления
- Commands – RoutedCommands
- Resources - словарь стилей

5. Модели данных

Модели - это классы которые описывают объекты предметной области. Каждая модель соответствует одной сущности системы.

Список моделей:

- Organization - организация (юридическое лицо)
- Warehouse - склад, принадлежащий организации
- Product - товар с характеристиками
- Category - категория товара
- Manufacturer - производитель
- Supplier - поставщик
- Order - накладная (поступление или списание)
- OrderItem - позиция накладной (товар + количество)

Пример модели Organization: `public`

```
class Organization
{     public static BigInteger OrgCounter =
BigInteger.Zero;
     public BigInteger OrgId { get; set;
}     public string OrgName { get; set; }
     public ObservableCollection<Warehouse> warehouses {
get;
set; }
          = new ObservableCollection<Warehouse>();
```

```
        public Organization(string
    name)
        {
            OrgId = ++OrgCounter;
            OrgName = name;
        }
    }
```

Обратите внимание на ObservableCollection<Warehouse> - это специальная коллекция WPF которая автоматически уведомляет интерфейс об изменениях. Когда добавляется новый склад - список в окне обновляется сам без ручного вызова.

Пример модели накладной с позициями:

```
    public class Order
    {      public BigInteger OrderId { get; set; }      public
    DateTime OrderDate { get; set; }        public OrderType
    OrderType { get; set; }     public BigInteger WarehouseId {
    get; set; }       public string Comment { get; set; } =
    string.Empty;     public List<OrderItem> Items { get; set; }
    = new
List<OrderItem>();
        public decimal
    TotalSum
        {         get         {             decimal sum = 0;
    foreach (var item in Items)                 sum +=
```

```
        item.PriceAtMoment * item.Quantity;              return
        sum;
                }
            }
        }
```

6. Контекст данных – ApplicationContext

ApplicationContext - это единое хранилище всех данных приложения. Все коллекции живут здесь. Сервисы получают контекст через конструктор и работают с его коллекциями.

```
        public class ApplicationContext
        {     public ObservableCollection<Organization>
        Organizations
{ get; set; }     public ObservableCollection<Warehouse>
        Warehouses { get;
set; }     public ObservableCollection<Product> Products { get;
        set;
}       public ObservableCollection<Order> Orders { get; set; }
        public ObservableCollection<Category> Categories { get;
set; }     public ObservableCollection<Manufacturer>
        Manufacturers
{ get; set; }     public ObservableCollection<Supplier> Suppliers
        { get;
set; }      public
        ApplicationContext()
            {
```

```
            Organizations = new
ObservableCollection<Organization>();
            Warehouses = new ObservableCollection<Warehouse>();
            FillTestData();
        }
    }
```

При запуске приложения создаётся один экземпляр ApplicationContext в App.xaml.cs и передаётся во все окна и сервисы. Это гарантирует что все части приложения работают с одними и теми же данными.

7. Сервисы

Сервисы - это классы с методами для работы с данными (CRUD). Они не знают про UI и работают только через ApplicationContext.

Список сервисов:

- ServiceOrgs - управление организациями
- ServiceWhs - управление складами
- ServiceProducts - управление товарами
- ServiceOrders - управление накладными

Каждый сервис получает контекст через конструктор:

```
public class ServiceOrgs
{     private readonly ApplicationContext
_context;
     public ServiceOrgs(ApplicationContext
context)
    {
        _context = context;
```

```
    }      public ObservableCollection<Organization>
GetAll()
    {          return
_context.Organizations;
    }      public void
Add(Organization org)
    {          if (org == null) return;          if
(string.IsNullOrWhiteSpace(org.OrgName)) return;
_context.Organizations.Add(org);

    }      public void Update(BigInteger id, string
newName)
    {          var org =
GetById(id);          if (org ==
null) return;
org.OrgName = newName;
    }      public void
Delete(BigInteger id)
    {          var org =
GetById(id);          if (org ==
null) return;
        _context.Organizations.Remove(org);
    }
}
```

Особого внимания заслуживает ServiceOrders - сервис накладных. Он содержит бизнес-логику изменения остатков товаров:

```
public (bool success, string error) CreateIncomeOrder(
    BigInteger warehouseId,
List<OrderItem> items,     string
comment = "")
{     if (items == null || items.Count ==
0)          return (false, "Добавьте хотя
бы одну позицию.");

     var wh = FindWarehouse(warehouseId);
if (wh == null)          return (false,
"Склад не найден.");
     var order = new Order(OrderType.Income, warehouseId,
comment);     order.Warehouse
= wh;
     foreach (var item in
items)
    {          item.Product.StockQuantity +=
item.Quantity;          if
(!wh.products.Contains(item.Product))
wh.products.Add(item.Product);

order.Items.Add(item);
    }
```

```
    _context.Orders.Add(order);
return (true, "");
}
```

Метод возвращает кортеж (bool success, string error) - это позволяет вернуть и результат операции и текст ошибки одним возвратом.

8. Unit-тесты

Каждый сервис покрыт Unit-тестами с использованием фреймворка xUnit. Тесты проверяют корректность работы каждого метода.

Тесты находящиеся в проекте WarehouseDataTests:

- ServiceOrgsTests.cs
- ServiceWhsTests.cs
- ServiceProductsTests.cs
- ServiceOrdersTests.cs

Пример тестов для ServiceOrders:

```
[Fact] public void
CreateIncomeOrder_IncreasesStock()
{     var (ctx, svcOrders, svcProducts) =
CreateSvc();     var product = ctx.Products[0];
var wh = ctx.Warehouses[0];     int stockBefore =
product.StockQuantity;
     var result =
svcOrders.CreateIncomeOrder(         wh.WhId,
MakeItems(product, 5));

    Assert.True(result.success);
```

```
        Assert.Equal(stockBefore + 5, product.StockQuantity);
    }
     [Fact] public void
    CreateOutcomeOrder_NotEnoughStock_ReturnsFalse()
    {
        var (ctx, svcOrders, svcProducts) = CreateSvc();
    var product = ctx.Products[0];     var wh =
    ctx.Warehouses[0];
         var     result     =     svcOrders.CreateOutcomeOrder(
    wh.WhId, MakeItems(product, product.StockQuantity +
999));

        Assert.False(result.success);
        Assert.NotEmpty(result.error);
    }
```

Атрибут [Fact] помечает метод как тест. Assert.True, Assert.Equal, Assert.False - проверяют ожидаемый результат.

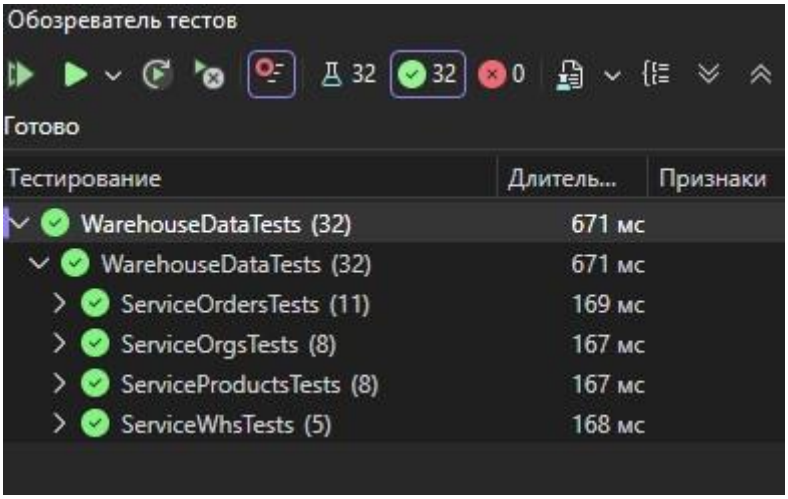


Рисунок 2 - Прохождение тестов

9. Пользовательский интерфейс

9.1 Единый стиль - ResourceDictionary

Все стили хранятся в файле DictionaryBasic.xaml и подключаются через App.xaml на уровне всего приложения:

```
<Application.Resources>
    <ResourceDictionary>
        <ResourceDictionary.MergedDictionaries>
            <ResourceDictionary
Source="Resources/DictionaryBasic.xaml"/>
        </ResourceDictionary.MergedDictionaries>
    </ResourceDictionary>
</Application.Resources>
```

В словаре описаны стили для всех стандартных элементов - кнопок, текстовых полей, таблиц, меню, строки состояния. Это гарантирует единый внешний вид всего приложения.

9.2 Навигация

Главное окно использует элемент Frame для навигации между модулями. Внутри Frame отображаются UserControl - самостоятельные блоки интерфейса:

```
<Frame x:Name="MainFrame"
       NavigationUIVisibility="Hidden"
       Background="White"/>
```

Переключение между модулями происходит через меню сверху:

```
private void MenuWarehouses_Click(object sender,
RoutedEventArgs e)
```

```
{     var whControl = new
UserControlWarehouses(ServiceWhs,
ServiceProducts, ServiceOrders);
whControl.WarehouseSelected += OnWarehouseSelected;
    MainFrame.Navigate(whControl);
}
```

9.3 RoutedCommands

Команды WPF позволяют привязать одно действие к нескольким элементам UI - кнопке, пункту меню и горячей клавише одновременно:

```
public static class WindowCommands
{     public static RoutedCommand OrgAdd { get; set; }
public static RoutedCommand OrgEdit { get; set; }
public static RoutedCommand OrgDelete { get; set; }
}
```

В XAML команда привязывается так:

```
<CommandBinding Command="local:WindowCommands.OrgAdd"
                Executed="OrgAdd_Executed"/>

<MenuItem Header="Новая организация"
          Command="local:WindowCommands.OrgAdd"/>
<Button Content="Добавить"
        Command="local:WindowCommands.OrgAdd"/>
```

10. Описание модулей

10.1 Модуль организаций

При запуске приложения открывается окно со списком организаций. Поддерживаются операции добавления, редактирования и удаления. Удаление реализовано с подтверждением - пользователь должен ввести название организации чтобы подтвердить удаление.

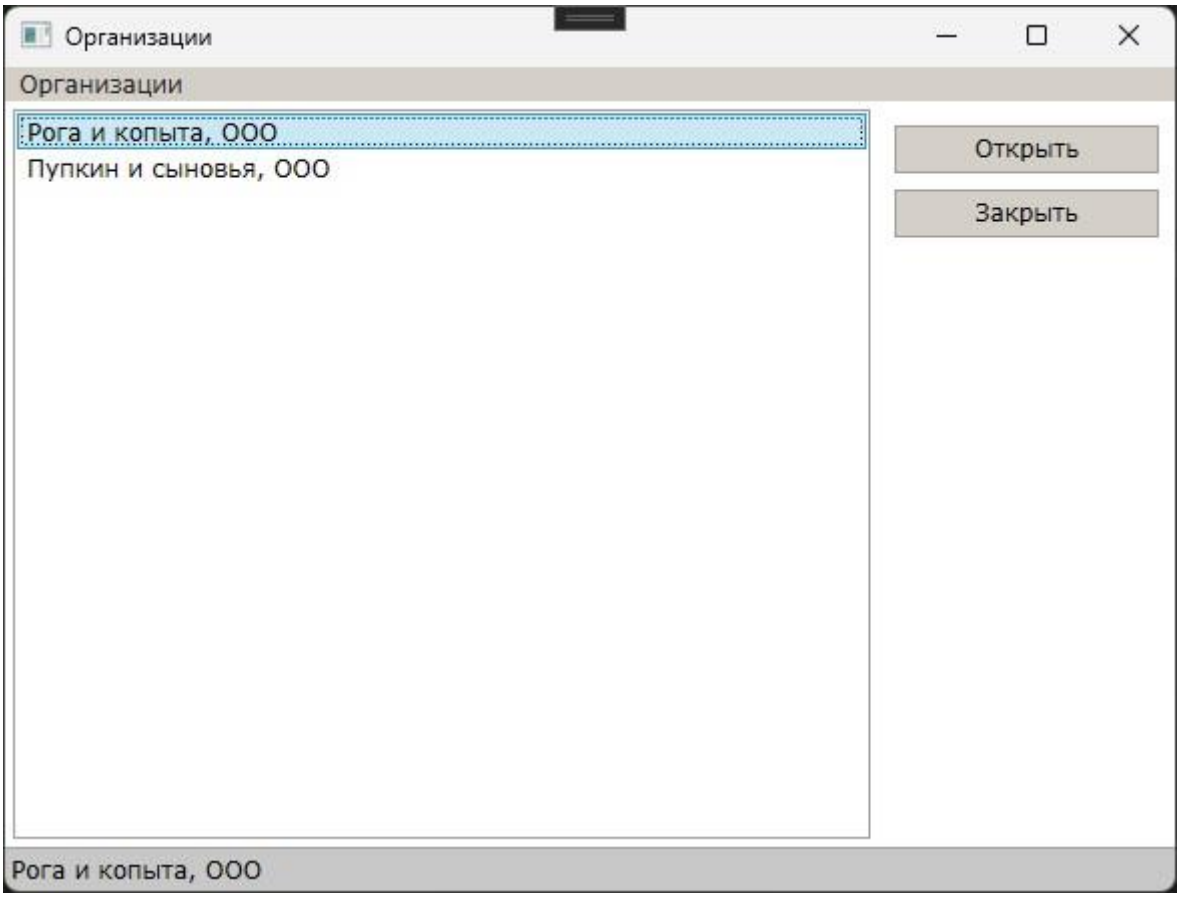


Рисунок 3 - Окно со списком организаций

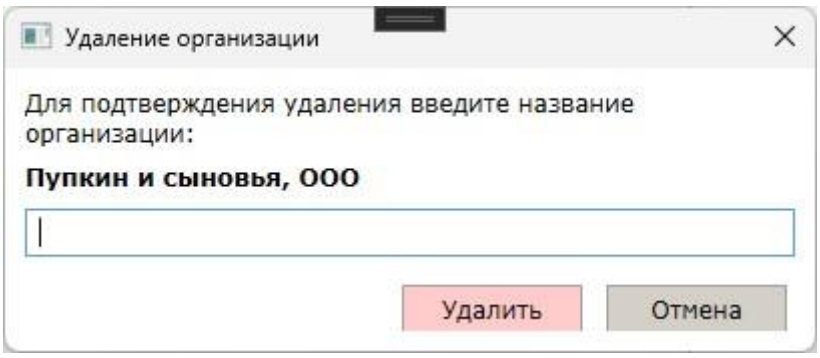


Рисунок 4 - Диалоговое окно подтверждения удаления организации

10.2 Модуль складов

После выбора организации открывается главное окно. Слева - список складов выбранной организации. Справа - товары выбранного склада в виде таблицы DataGrid.

Строка состояния внизу показывает текущий контекст - название организации и выбранного склада:

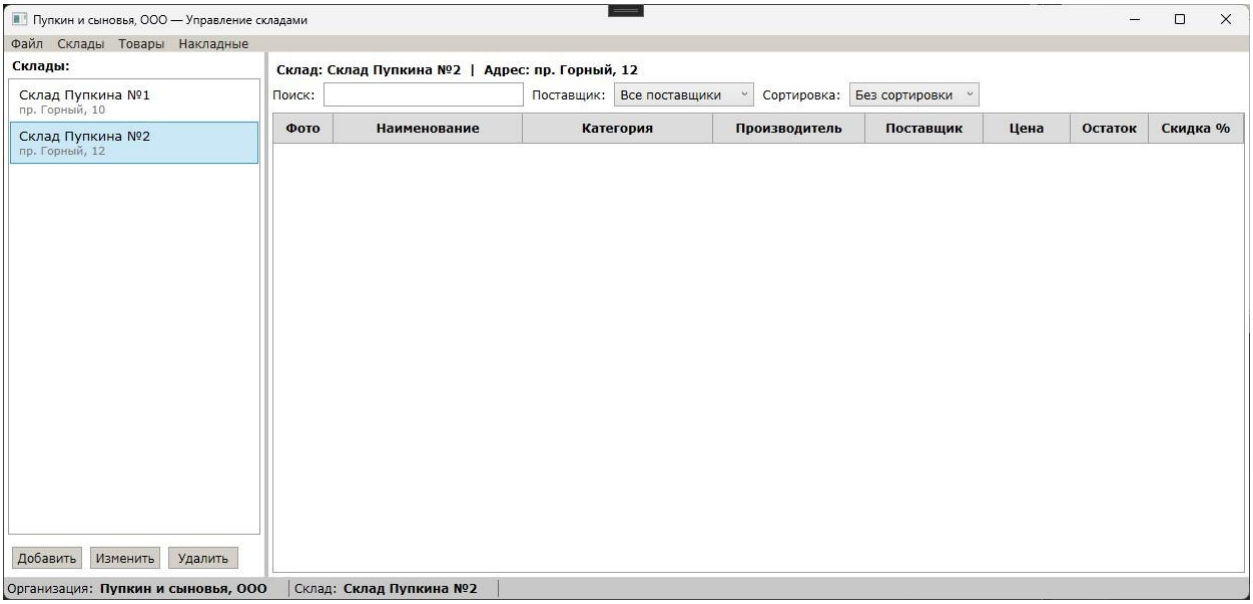


Рисунок 5 - Окно управления складами

10.3 Модуль товаров

Товары отображаются в таблице DataGrid с колонками: Фото, Наименование, Категория, Производитель, Поставщик, Цена, Остаток, Скидка %.

Реализованы поиск по наименованию, фильтрация по поставщику и сортировка по клику на заголовок колонки.

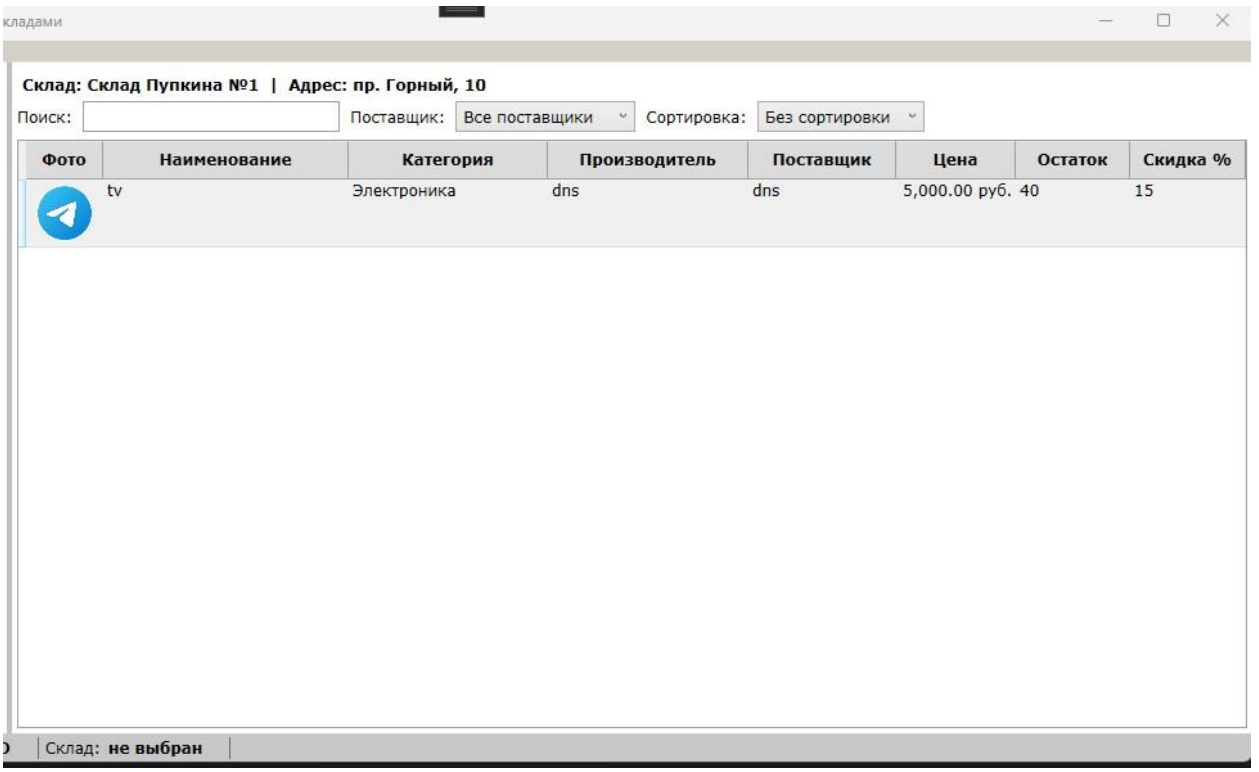


Рисунок 6 - Таблица товаров с данными

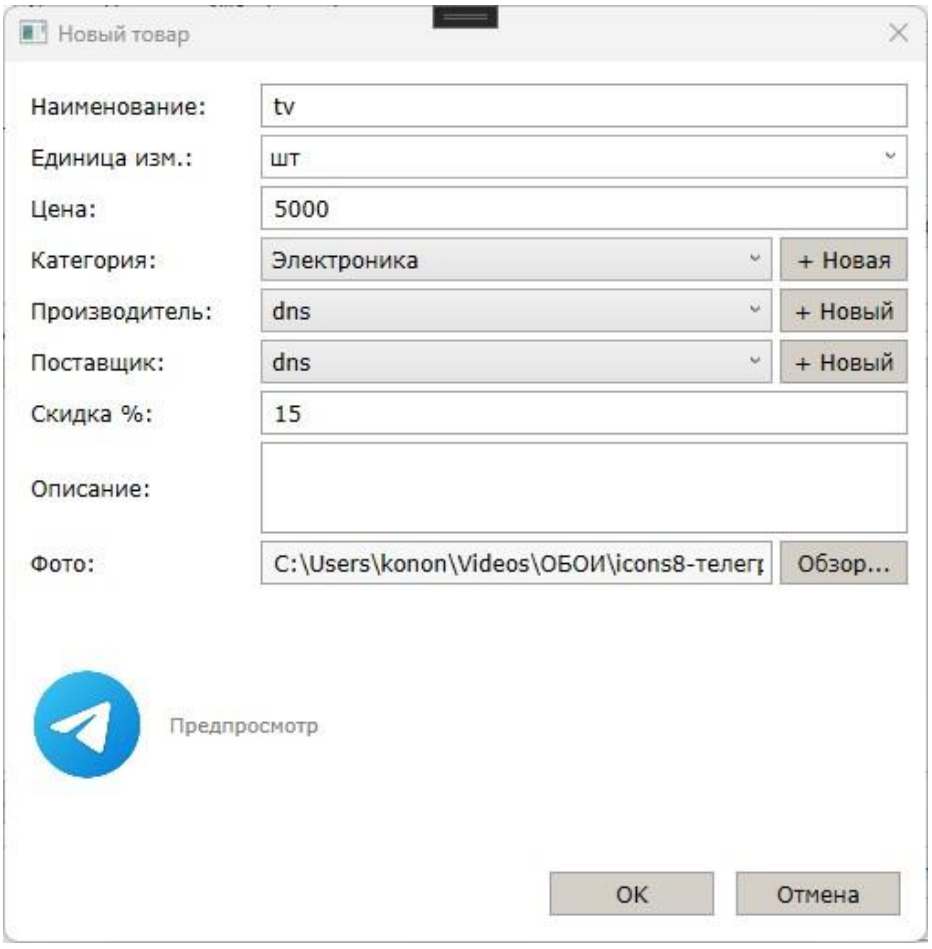


Рисунок 7 - форма добавления/редактирования товара

10.4 Модуль накладных

Накладная - основной документ системы. Через неё происходит всё движение товаров - поступление на склад и списание со склада.

Форма накладной позволяет:

- Выбрать тип - поступление или списание
- Добавить несколько позиций (товар + количество)
- Создать новый товар прямо из формы накладной
- Видеть итоговую сумму накладной

После проведения накладной остатки товаров автоматически изменяются.

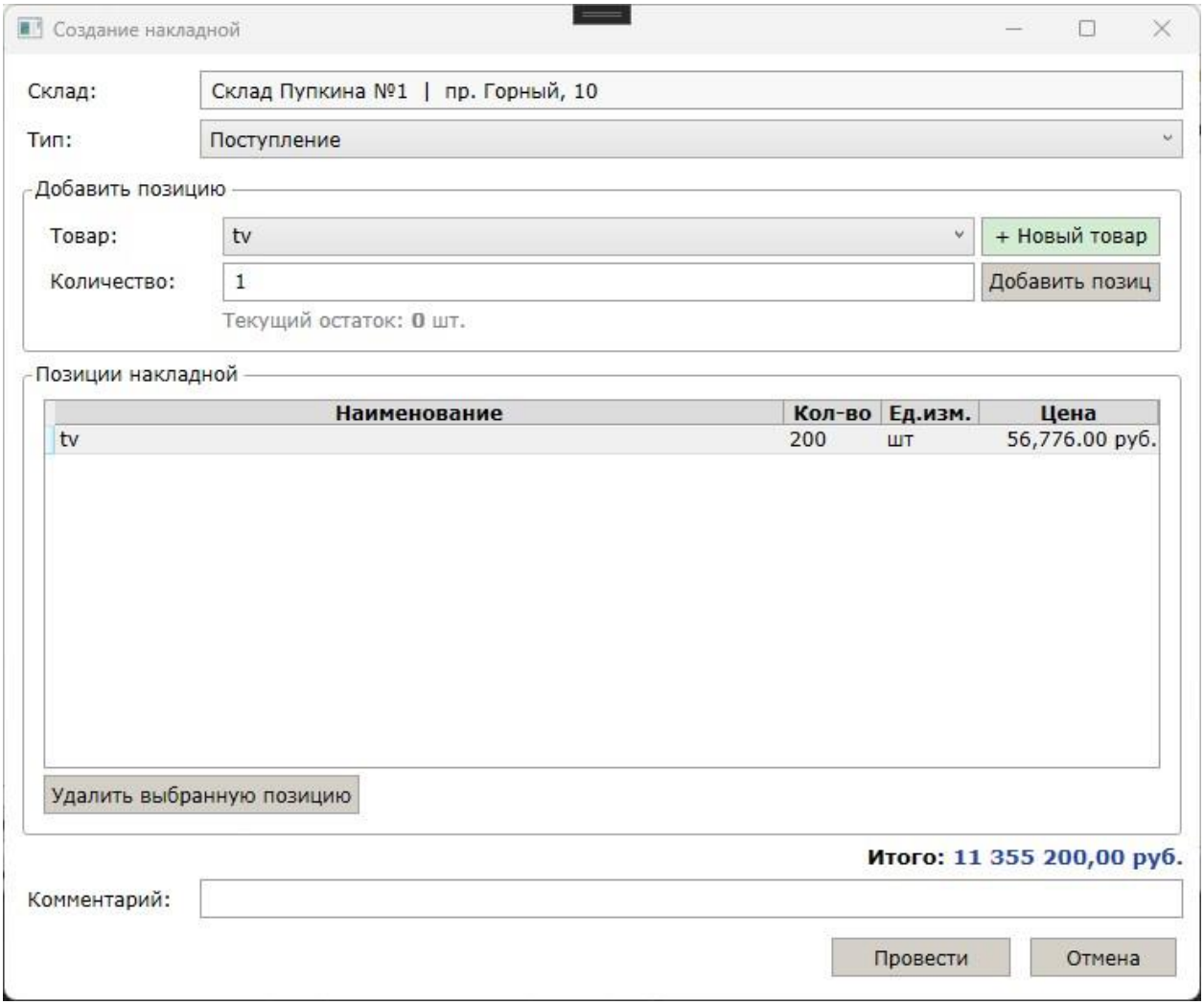


Рисунок 8 - Форма создания накладной

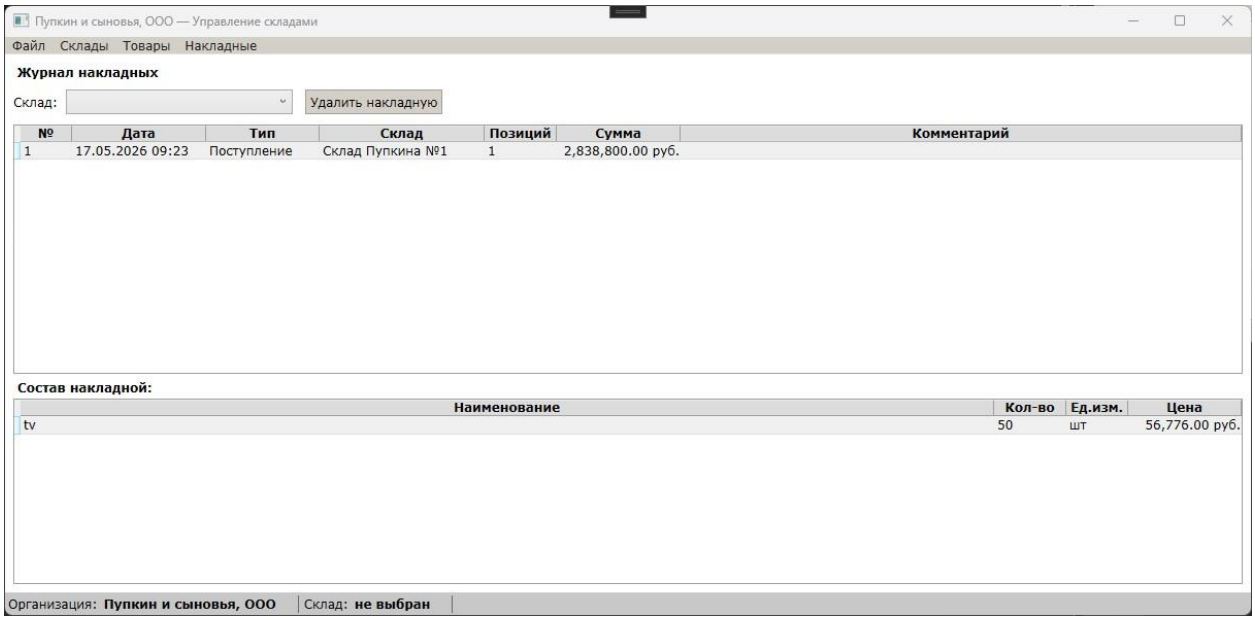


Рисунок 8 - Журнал накладных

11. Валидация данных

Все формы ввода содержат валидацию на стороне клиента. При некорректном вводе появляется сообщение об ошибке.

Примеры проверок:

- Поле "Наименование" не может быть пустым
- Цена должна быть числом больше или равным 0
- Скидка должна быть от 0 до 100
- При списании проверяется что остатка хватает
- При удалении организации или склада пользователь должен ввести название для подтверждения

```
if (string.IsNullOrWhiteSpace(name))
{
    MessageBox.Show("Введите наименование товара.",
"Ошибка",
```

```
            MessageBoxButton.OK, MessageBoxImage.Warning);
    tbName.Focus();     return;
    }  if (!decimal.TryParse(tbPrice.Text, out decimal price) ||
price < 0)
    {
        MessageBox.Show("Введите корректную цену.", "Ошибка",
            MessageBoxButton.OK, MessageBoxImage.Warning);
    tbPrice.Focus();     return;
    }
```

12. Хранение данных

Данные хранятся в памяти через ObservableCollection без использования базы данных. Это соответствует требованиям учебного задания.

ObservableCollection реализует интерфейс INotifyCollectionChanged - при добавлении или удалении элемента коллекция автоматически уведомляет все привязанные к ней элементы UI. Благодаря этому списки и таблицы обновляются мгновенно без ручного вызова методов обновления.

Заключение

В ходе выполнения работы было разработано десктоп-приложение «Управление складом» на языке C# с использованием технологии WPF. Приложение реализует основные процессы складского учёта: работу с организациями и складами, ведение товарной номенклатуры, а также оформление приходных и расходных накладных.

Архитектура решения построена в виде трёх отдельных проектов: библиотека бизнес-логики WarehouseData, WPF-приложение Warehouse и проект автоматического тестирования WarehouseDataTests. Такое разделение обеспечивает независимость пользовательского интерфейса и логики приложения, упрощает сопровождение и развитие системы.

В приложении были реализованы основные сущности предметной области и сервисы для работы с данными. Для хранения информации использовались коллекции ObservableCollection, благодаря которым интерфейс автоматически обновляется при изменении данных. Бизнес-логика изменения остатков товаров вынесена в отдельные сервисные классы.

Для проверки корректности работы системы был создан набор Unitтестов на базе xUnit. Тестирование охватывает операции добавления, изменения и удаления данных, а также логику проведения накладных и изменения товарных остатков.

Пользовательский интерфейс разработан в классическом desktop-стиле с применением стандартных элементов WPF и механизма ResourceDictionary для единого оформления приложения. Навигация между модулями реализована через Frame и UserControl, а действия пользователя - через RoutedCommands. Также в приложении реализована валидация вводимых данных и защита от случайного удаления записей.

Таким образом, все требования технического задания были выполнены. В ходе работы были применены современные подходы к разработке программного обеспечения: разделение приложения на слои, использование сервисной архитектуры, автоматическое тестирование и построение интерфейса на XAML. Полученные навыки разработки WPF-приложений и проектирования архитектуры являются важными для создания корпоративных приложений на платформе .NET.